

MANAGEMENT INFORMATION SYSTEM

Unit-IV: Detailed System Design

Comprehensive Study Notes

1. AIM OF DETAILED SYSTEM DESIGN

1.1 Introduction to Detailed System Design

Definition: Detailed System Design is the phase of system development where the conceptual design is transformed into a detailed, precise specification that can be implemented by programmers and technical teams.

Purpose: To provide complete and accurate specifications for building the system, covering all technical aspects including inputs, outputs, processes, databases, and interfaces.

1.2 Detailed Design vs Conceptual Design

Aspect	Conceptual Design	Detailed Design
Purpose	What system will do	How system will do it
Level	High-level	Low-level, specific
Audience	Management, users	Technical team
Output	Design report	Technical specifications
Detail	Overview	Complete details
Decision	Feasibility, scope	Implementation
Example	"System will track inventory"	"Inventory table with fields: ItemID, Name, Qty, Price..."

1.3 Key Deliverables of Detailed Design

Deliverable	Description
System Specifications	Complete system description
Database Design	Tables, relationships, schemas
Input Design	Screens, forms, data entry
Output Design	Reports, displays, dashboards
Process Design	Algorithms, workflows
User Interface Design	Screens, navigation
Technical Architecture	Hardware, software, network
Test Plans	Testing strategies and cases
Documentation	User and technical manuals

2. PROJECT MANAGEMENT

2.1 Introduction to Project Management

Definition: Project Management is the application of knowledge, skills, tools, and techniques to project activities to meet project requirements and objectives.

2.2 Importance of Project Management in MIS Development

Reason	Description
Scope Control	Prevents scope creep
Timely Delivery	Ensures project meets deadlines
Budget Control	Prevents cost overruns
Quality Assurance	Ensures quality standards
Risk Management	Identifies and mitigates risks
Resource Optimization	Efficient use of resources
Stakeholder Communication	Keeps everyone informed

2.3 Project Management Knowledge Areas

Knowledge Area	Description
Scope Management	Defining and controlling what is included in the project
Time Management	Scheduling activities, milestones, and deadlines
Cost Management	Budgeting, estimating, and controlling costs
Quality Management	Ensuring deliverables meet quality standards
HR Management	Managing project team and resources
Communication Management	Information distribution to stakeholders
Risk Management	Identifying, analyzing, and mitigating risks
Procurement Management	Acquiring goods and services from external sources

2.4 Project Management Phases

Phase	Key Activities	Outputs
Initiation	Define project, identify stakeholders, develop charter	Project charter
Planning	Define scope, create schedule, budget, risk plan	Project management plan
Execution	Build the system, manage team, communicate	Deliverables, work results
Monitoring & Control	Track progress, manage changes, quality control	Performance reports
Closure	Final delivery, documentation, lessons learned	Project closure report

2.5 Project Management Methodologies

A. Waterfall Model

Characteristics: Sequential phases - each phase completed before next; documentation at each stage; suitable for well-defined projects.

Aspect	Description
Advantages	Simple to understand, structured approach, comprehensive documentation, clear milestones
Disadvantages	Inflexible, late feedback, testing happens at the end, not suitable for complex projects

B. Agile Methodology

Characteristics: Iterative development, continuous feedback, flexible to changes, regular delivery.

Aspect	Description
Advantages	Flexibility, customer involvement, faster delivery, continuous testing
Disadvantages	Less documentation, requires customer involvement, may not fit fixed-price contracts

C. Comparison: Waterfall vs Agile

Aspect	Waterfall	Agile
Approach	Sequential	Iterative
Flexibility	Low	High
Documentation	Extensive	Minimal
Customer Involvement	Low	High
Testing	At the end	Throughout
Risk	Higher	Lower
Best For	Well-defined projects	Complex, changing projects

2.6 Project Management Tools

Tool Category	Examples	Purpose
Project Planning	MS Project, Jira	Scheduling, tracking
Collaboration	Slack, Teams	Communication
Documentation	Confluence, Google Docs	Document sharing
Version Control	Git, SVN	Code management
Testing	JIRA, TestRail	Test management
Reporting	Power BI, Excel	Progress reporting

3. DEFINE SUBSYSTEM

3.1 Introduction to Subsystems

Definition: A subsystem is a self-contained component of a larger system that performs a specific function. Breaking a large system into subsystems makes it easier to design, develop, and manage.

3.2 Why Divide into Subsystems?

Reason	Description
Complexity Management	Easier to handle smaller pieces
Parallel Development	Different teams work on different subsystems
Easier Testing	Test subsystems independently
Modularity	Subsystems can be reused
Maintenance	Easier to fix and update
Flexibility	Subsystems can be changed independently

3.3 Types of Subsystems

Type	Description	Example
Functional Subsystem	Performs a specific business function	Order processing, Inventory management
Technical Subsystem	Provides technical services	Database, Security, Network
Support Subsystem	Supports other subsystems	Reporting, Logging, Backup

3.4 Subsystem Interface Definition

Interface Element	Description
Input/Output	What data is exchanged
Format	Data structure, message format
Protocol	How communication happens
Frequency	How often they interact
Synchronization	Synchronous vs asynchronous

4. INPUT, OUTPUT & PROCESS DESIGN

4.1 INPUT DESIGN

4.1.1 Introduction

Definition: Input design is the process of defining how data will be entered into the system, including the format, screens, validation rules, and user interfaces.

4.1.2 Objectives of Input Design

Objective	Description
Accuracy	Ensure data entered is correct
Efficiency	Minimize time and effort for data entry
User-Friendliness	Easy to use and understand
Validation	Catch errors at entry point
Consistency	Uniform data format
Security	Prevent unauthorized access

4.1.3 Input Design Guidelines

- Minimize Input - Only collect necessary data
- Provide Defaults - Auto-fill common values
- Group Related Fields - Logical grouping of fields
- Use Drop-down Lists - Reduce typing errors
- Provide Validation - Check data on entry
- Use Clear Labels - Easy to understand
- Provide Help - Context-sensitive help
- Tab Order - Logical navigation
- Error Messages - Clear and constructive

4.1.4 Input Validation Techniques

Technique	Description	Example
Presence Check	Field cannot be empty	Name is required
Format Check	Follows required pattern	Email: x@y.com
Range Check	Within specified range	Age: 18-60
List Check	Value in allowed list	Gender: M/F/O
Format Mask	Fixed format	Phone: (XXX) XXX-XXXX
Length Check	Minimum/Maximum length	Password: 8+ chars
Type Check	Correct data type	Date must be valid date

4.1.5 Input Devices

Device	Purpose	Best For
Keyboard	Text entry	All data entry
Mouse/Touch	Selection	Menu, checkbox
Scanner	Document capture	Forms, images
Barcode Scanner	Product/ID capture	Inventory, POS
Touch Screen	Direct input	Public kiosks
Voice Input	Speech recognition	Hands-free
Biometric	Identity verification	Security, attendance

4.1.6 Input Screen Design Example

+-----+

PRODUCT ENTRY FORM

+-----+

Product ID: [] Status: [Active ▼]

Product Name: [_____]
Category: [Electronics ▼]
Price: [\$_____] Currency: [USD ▼]
Quantity: [_____] Unit: [Pcs ▼]
Description: [_____] [_____]
[Save] [Cancel] [Clear]
+-----+

4.2 OUTPUT DESIGN

4.2.1 Introduction

Definition: Output design defines how information will be presented to users, including reports, screens, dashboards, and other display formats.

4.2.2 Objectives of Output Design

Objective	Description
Relevance	Provide needed information
Clarity	Easy to understand
Timeliness	Available when needed
Accuracy	Correct information
Format	Appropriate presentation
User-Friendly	Easy to use and interpret
Security	Confidential when required

4.2.3 Report Types

Type	Description	Frequency	Example
Scheduled Report	Regular intervals	Daily, Weekly, Monthly	Daily Sales Report
Exception Report	Only when unusual	As needed	Low Stock Alert
Demand Report	User requested	On request	Custom Query
Summary Report	Consolidated data	Periodic	Monthly Summary
Detail Report	All transactions	Regular	Transaction List
Dashboard	Visual summary	Real-time	Executive Dashboard

4.2.4 Output Design Guidelines

- Identify Users - Who needs the output
- Define Purpose - Why it is needed
- Determine Content - What information to include
- Choose Format - Appropriate presentation
- Consider Frequency - How often it's needed
- Design Layout - Easy to read and understand
- Include Headers - Titles, dates, page numbers
- Use Summaries - Totals, averages
- Highlight Exceptions - Draw attention to issues

4.2.5 Sample Report Layout

+-----+ MONTHLY SALES REPORT March 2024 +-----+				
	REGION	SALES	TARGET	% ACHIEVED

	North	& 45,000	& 50,000	90%
	South	& 52,000	& 50,000	104%

East	& 38,000	& 40,000	95%	
West	& 48,000	& 45,000	107%	

TOTAL	& 183,000	& 185,000	99%	
TOP PERFORMER: West Region (107%)				
NEEDS ATTENTION: East Region (95%)				
Generated: 01-Apr-2024			Page 1 of 1	
+-----+				

4.2.6 Dashboard Design

Definition: A dashboard is a visual display of key performance indicators (KPIs) and metrics, presented in an easy-to-read format.

Element	Description	Example
KPIs	Key performance indicators	Revenue, Orders
Charts	Visual data representation	Bar, Line, Pie
Metrics	Numerical measures	Count, Percentage
Alerts	Exception notifications	Low stock alert
Trends	Historical patterns	Sales trend
Filters	Data selection	Date range, Region

4.3 PROCESS DESIGN

4.3.1 Introduction

Definition: Process design defines how data is transformed from input to output, including the algorithms, workflows, business rules, and processing logic.

4.3.2 Process Design Techniques

A. Flowcharts

A flowchart is a diagram that shows the steps in a process using standard symbols (start/end, process, decision, input/output).

B. Data Flow Diagrams (DFD)

DFD shows how data flows through the system, highlighting inputs, processes, outputs, and storage.

Symbol	Name	Description
○	Process	Transforms data
□	External Entity	Source/Destination of data
→	Data Flow	Movement of data
+	Data Store	Storage of data

DFD Level	Description
Level 0 - Context Diagram	Single process, external entities
Level 1 - Major Processes	Sub-processes, major functions
Level 2 - Detailed Processes	Specific processes and data flows

C. Structured English

A method of describing process logic using structured statements in plain English.

D. Decision Tables

A table that shows the relationship between conditions and actions.

Condition	Rule 1	Rule 2	Rule 3	Rule 4
Order > & 10,000	Y	Y	N	N
VIP Customer	Y	N	Y	N
Action: 10% Discount	✓			
Action: 5% Discount		✓	✓	
Action: No Discount				✓

E. Pseudocode

A simplified programming language that describes the process logic.

5. DATABASE DESIGN

5.1 Introduction

Definition: Database design is the process of structuring data to be stored in a database, including defining tables, relationships, constraints, and indexes.

5.2 Entity-Relationship (ER) Diagrams

Definition: An ER diagram is a visual representation of entities (objects), their attributes, and the relationships between them. Entities are represented as rectangles, attributes as ellipses, and relationships as diamonds.

5.3 Normalization

Normal Form	Description	Rule
1NF (First Normal Form)	Eliminate duplicate columns, create separate tables for related data	Each cell contains a single value
2NF (Second Normal Form)	Remove partial dependencies	All non-key attributes depend on the full primary key
3NF (Third Normal Form)	Remove transitive dependencies	No non-key attribute depends on another non-key attribute

5.4 Database Tables Example

Table	Fields	Primary Key
Customer	CustomerID, Name, Email, Phone, Address	CustomerID
Product	ProductID, Name, Category, Price, StockQty	ProductID
Order	OrderID, CustomerID, OrderDate, TotalAmount, Status	OrderID
OrderItem	OrderItemID, OrderID, ProductID, Quantity, Price	OrderItemID

5.5 Relationships

Relationship	Description	Example
One-to-One (1:1)	One entity relates to one other entity	Customer ↔ Login Credentials
One-to-Many (1:M)	One entity relates to many others	Customer → Orders
Many-to-Many (M:N)	Many entities relate to many others	Student ↔ Course

6. USER INTERFACE DESIGN

6.1 Introduction

Definition: User Interface (UI) Design is the process of designing the visual elements of a system that users interact with, including screens, menus, buttons, and navigation.

6.2 GUI vs Character-Based Interface

Aspect	GUI (Graphical User Interface)	Character-Based Interface
Visual Elements	Graphics, icons, windows	Text, characters
Navigation	Mouse, touch, clicks	Keyboard commands
Ease of Use	Intuitive, user-friendly	Requires training
Flexibility	Rich interaction	Limited interaction
Resource Usage	Higher memory/processing	Low resource usage
Example	Windows, macOS, Android	Linux terminal, MS-DOS

6.3 Principles of UI Design

Principle	Description
Consistency	Same actions have same results; uniform layout throughout
Feedback	System provides immediate response to user actions
Error Prevention	Design that prevents errors from occurring
User Control	Users can undo actions, navigate freely
Flexibility	Accommodates different user skill levels (shortcuts, customization)
Visibility	All options and actions are clearly visible
Simplicity	Minimalist design, only necessary elements
Tolerance	System handles errors gracefully

7. SYSTEM TESTING

7.1 Introduction

Definition: System testing is the process of verifying that a system meets its requirements and works correctly in all expected conditions.

7.2 Objectives of Testing

Objective	Description
Verification	Is the system built correctly?
Validation	Is the right system built?
Error Detection	Find bugs and defects
Quality Assurance	Ensure quality standards
Risk Reduction	Minimize failure risk
User Satisfaction	Meet user expectations

7.3 Testing Types

A. Unit Testing

Aspect	Description
What	Individual components
Who	Developers
When	During development
Purpose	Verify each unit works

B. Integration Testing

Aspect	Description
What	Combined components
Who	Developers/Testers
When	After unit testing
Purpose	Verify components work together

C. System Testing

Aspect	Description
What	Complete system
Who	Testers
When	Before UAT
Purpose	Verify full system

D. User Acceptance Testing (UAT)

Aspect	Description
What	Complete system
Who	End users
When	Before deployment
Purpose	Verify meets user needs

E. Performance Testing

Type	Description
Load Testing	Normal expected load
Stress Testing	Beyond normal capacity
Volume Testing	Large data volumes
Endurance Testing	Extended use

F. Security Testing

Verifying security controls, access control, data protection, and resistance to unauthorized access.

G. Regression Testing

Re-testing existing functionality after changes to ensure nothing is broken.

7.4 Test Case Template

Field	Description	Example
Test Case ID	Unique ID	TC-001
Test Case Name	Name	Valid Order Creation
Description	What's being tested	Create order with valid data
Pre-conditions	Setup required	User logged in, items in cart
Test Data	Data required	Customer ID: C001, Items: P001 x 2
Steps	How to execute	1. Add items to cart 2. Enter payment info 3. Submit order
Expected Result	What should happen	Order created, confirmation sent
Actual Result	What happened	Order created (PASS)
Status	Pass/Fail	PASS

8. SOFTWARE & HARDWARE SELECTION

8.1 Introduction

Definition: The process of selecting the appropriate hardware, software, and network infrastructure required to implement the designed system.

8.2 Software Selection Criteria

Criteria	Description
Functionality	Meets requirements
Compatibility	Works with existing systems
Scalability	Can grow with needs
Performance	Speed and reliability
Cost	Purchase and maintenance
Support	Vendor support availability
User Friendliness	Easy to use
Security	Security features

8.3 Software Selection Process

1. Identify Requirements - What software must do
2. Research Options - Find available software
3. Create Shortlist - Top 3-5 candidates
4. Evaluate Shortlist - Compare features, cost
5. Vendor Demo - See software in action
6. Compare Proposals - Final evaluation
7. Select Software - Make final decision
8. Negotiate & Purchase

8.4 Hardware Selection Criteria

Criteria	Description
Performance	Speed, power
Capacity	Storage, memory
Reliability	Uptime, durability
Scalability	Can grow
Compatibility	Works with software
Support	Vendor support
Cost	Purchase, maintenance
Warranty	Protection

8.5 Make-or-Buy Decision

Factor	Build	Buy	Customize
Requirements	Unique	Standard	Near-standard
Cost	Higher	Lower	Medium
Time	Longer	Quick	Medium
Control	Full	Limited	Some
Risk	Higher	Lower	Medium
Maintenance	Internal	Vendor	Hybrid

9. SYSTEM IMPLEMENTATION

9.1 Introduction

Definition: System implementation is the phase where the designed system is actually built, tested, and deployed into the production environment.

9.2 Training Methods

Method	Description	Advantages	Disadvantages
Individual Training	One-on-one coaching	Personalized attention	Time-consuming, expensive
Group Training	Classroom-style sessions	Cost-effective, standardized	Less personalized
Computer-Based Training	Self-paced e-learning modules	Flexible schedule, consistent content	Lacks human interaction

9.3 Testing Levels (Summary)

Level	Scope	Performed By
Unit Testing	Individual modules	Developers
Integration Testing	Combined modules	Developers/Testers
System Testing	Complete system	Testers
Acceptance Testing	User requirements	End users

9.4 Conversion Strategies

Strategy	Description	Advantages	Disadvantages
Direct Conversion	Old system stopped, new system started immediately	Quick, simple	High risk, no fallback
Parallel Conversion	Both systems run simultaneously	Low risk, easy comparison	Expensive, double work
Pilot Conversion	New system implemented in one location first	Limited risk, test in real environment	Slower rollout
Phased Conversion	New system implemented in stages	Manageable, gradual transition	Complex coordination

10. SYSTEM MAINTENANCE

10.1 Introduction

Definition: System maintenance is the process of modifying a system after its implementation to correct issues, improve performance, or adapt to changing requirements.

10.2 Types of Maintenance

Type	Description	Example
Corrective Maintenance	Fixing bugs and errors discovered after implementation	Fixing calculation errors, crash bugs
Adaptive Maintenance	Modifying system to adapt to changes in environment	Upgrading for new OS, new regulations
Perfective Maintenance	Improving system performance, usability, or adding features	Adding new reports, optimizing queries
Preventive Maintenance	Proactive changes to prevent future problems	Database indexing, code refactoring

11. SYSTEM EVALUATION

11.1 Introduction

Definition: System evaluation is the process of assessing a system's performance, effectiveness, and value after implementation to determine whether it meets its objectives and delivers expected benefits.

11.2 Evaluation Criteria

Criteria	Description
System Performance	Speed, response time, throughput, uptime
User Satisfaction	User feedback, ease of use, training adequacy
Business Impact	ROI, cost savings, productivity improvement
Data Quality	Accuracy, completeness, consistency of data
Security	Effectiveness of access controls, breach incidents
Maintainability	Ease of making changes, fixing issues

12. DOCUMENTATION OF DETAILED DESIGN

12.1 Introduction

Definition: Documentation is the process of creating comprehensive written records of the system design, specifications, and instructions for use and maintenance.

12.2 Importance of Documentation

Importance	Description
Reference	Guide for developers
Communication	Share knowledge
Training	User and staff training
Maintenance	Fix and update system
Audit	Compliance and review
Historical Record	Track decisions
Knowledge Transfer	When staff changes

12.3 Types of Documentation

A. System Documentation

Document	Content
System Design Document (SDD)	System architecture, database design, input/output design, process design, interface design, security design, test plan
Database Documentation	ER diagrams, table definitions, relationships, constraints, indexes, views, stored procedures
Source Code Documentation	Header comments, function comments, inline comments, README files, API documentation

B. User Documentation

Type	Purpose	Content
User Manual	Guide for end users	Getting started, core functions, screens, reports, FAQs, troubleshooting
Quick Reference Guide	Short, easy-to-use reference	Key shortcuts, common tasks, important links
Online Help	Context-sensitive assistance	Tooltips, video tutorials, FAQ

C. Technical Documentation

Type	Content
Technical Reference Manual	System architecture, database schema, file structure, programming standards, error codes, backup/recovery procedures
API Documentation	Endpoints, methods, parameters, authentication, response format, error codes

D. Operations Documentation

Type	Content
Operations Manual	System startup/shutdown procedures, backup schedules, monitoring, incident response
Administration Guide	User management, security settings, system configuration, performance tuning

13. KEY DIAGRAMS FOR EXAM

Diagram	Description
Project Management Process	Initiation → Planning → Execution → Monitoring → Closure
Waterfall Model	Sequential phases
Agile Model	Iterative sprints
Subsystem Structure	System divided into subsystems
Input Design	Data entry screens
Output Types	Scheduled, On-demand, Exception, Dashboard
Process Flowchart	Steps in a process
DFD Example	Data flow diagram
Testing Types	Unit, Integration, System, UAT
Software Selection Process	Requirements → Research → Evaluate → Select
Documentation Types	System, User, Technical, Operational

14. IMPORTANT DEFINITIONS

Term	Definition
Detailed Design	Low-level system design with implementation details
Subsystem	Self-contained component of a larger system
Input Design	Defining data entry methods and screens
Output Design	Defining information presentation formats
Process Design	Defining data transformation logic
Testing	Verifying system works correctly
Unit Testing	Testing individual components
Integration Testing	Testing combined components
UAT	User Acceptance Testing
Software Selection	Choosing appropriate software
Hardware Selection	Choosing appropriate hardware
Documentation	Creating system records
SDD	System Design Document
DFD	Data Flow Diagram
ER Diagram	Entity-Relationship Diagram
Normalization	Organizing data to reduce redundancy

15. PREVIOUS YEAR QUESTIONS

Long Questions (10 marks)

- | Explain the aim and objectives of detailed system design.
- | What is project management? Explain the project management process.
- | Compare Waterfall and Agile methodologies.
- | Explain the concept of subsystems with examples.
- | Describe the process of input design with guidelines and validation techniques.
- | Explain output design with types of reports and sample report formats.
- | What is process design? Explain various process design techniques.
- | Describe the different types of system testing.
- | Explain the software and hardware selection process.
- | What is documentation? Explain different types of system documentation.

Short Questions (5 marks)

- | What is the difference between conceptual and detailed design?
- | What is a subsystem? Why are systems divided into subsystems?
- | Explain any two input validation techniques.
- | What is a dashboard? Give examples.
- | What is the difference between scheduled and exception reports?
- | Explain any two process design techniques.

What is the difference between unit testing and integration testing?

What is User Acceptance Testing (UAT)?

What are the criteria for software selection?

What is the purpose of system documentation?

Very Short Questions (2 marks)

What is detailed system design?

What is a subsystem?

What is input design?

What is output design?

What is a flowchart?

What is DFD?

What is unit testing?

What is UAT?

What is the make-or-buy decision?

What is a System Design Document?

16. SUMMARY OF UNIT IV

Topic	Key Points
Detailed Design Aim	Convert conceptual design to implementation specs
Project Management	Initiation, Planning, Execution, Monitoring, Closure
Methodologies	Waterfall (sequential), Agile (iterative)
Subsystems	Modular decomposition of system
Input Design	Data entry methods, validation, user-friendly forms
Output Design	Reports, dashboards, user presentations
Process Design	Flowcharts, DFD, Structured English, Pseudocode, Decision Tables
Database Design	ER diagrams, normalization, tables, relationships
UI Design	GUI vs character-based, consistency, feedback, error prevention
Testing	Unit, Integration, System, UAT, Performance, Security
Software Selection	Requirements, research, evaluation, purchase
Hardware Selection	Performance, capacity, reliability, cost
Implementation	Training, testing, conversion strategies
Maintenance	Corrective, Adaptive, Perfective, Preventive
Documentation	System, User, Technical, Operational